

# DATA STRUCTURES

## Unit-1: Introduction to Data Structures

Comprehensive Study Notes

# 1. What is a Data Structure?

---

A **data structure** is a way of organizing, storing, and managing data so that it can be accessed and modified efficiently. In simple terms, it is a structured arrangement of data in memory.

## Need for Data Structures

---

- Efficient data management and organization
- Faster access and modification of data
- Optimized memory usage
- Reusability and modularity in programs
- Simplifies complex problem-solving

## Characteristics of Data Structures

---

| Characteristic  | Description                                                  |
|-----------------|--------------------------------------------------------------|
| Linear          | Data items arranged in a sequence (e.g., Array, Linked List) |
| Non-Linear      | Data items not in a sequence (e.g., Tree, Graph)             |
| Homogeneous     | All elements of the same type (e.g., Array)                  |
| Non-Homogeneous | Elements may be of different types (e.g., Structures)        |
| Static          | Size fixed at compile time (e.g., Array)                     |
| Dynamic         | Size can change during execution (e.g., Linked List)         |

## 2. Types of Data Structures

```
DATA STRUCTURES
|
+-- PRIMITIVE: Integer, Character, Boolean, Float, Pointer
|
+-- NON-PRIMITIVE (ABSTRACT)
    |
    +-- LINEAR
        |
        +-- Array
        |
        +-- Stack
        |
        +-- Queue
        |
        +-- Linked List
    +-- NON-LINEAR
        +-- Tree
        +-- Graph
```

### Linear Data Structures

Elements are arranged in a sequential order, one after another. Each element has a unique predecessor and successor (except first and last).

| Data Structure | Description                                                               |
|----------------|---------------------------------------------------------------------------|
| Array          | Collection of elements of same type stored in contiguous memory locations |
| Stack          | Follows LIFO (Last In First Out) principle                                |
| Queue          | Follows FIFO (First In First Out) principle                               |
| Linked List    | Elements stored in nodes connected via pointers                           |

### Non-Linear Data Structures

Elements are not arranged sequentially; each element may connect to multiple other elements.

| Data Structure | Description                                            |
|----------------|--------------------------------------------------------|
| Tree           | Hierarchical structure with parent-child relationships |
| Graph          | Set of nodes (vertices) connected by edges             |

## 3. Basic Operations on Data Structures

---

| Operation | Description                                 |
|-----------|---------------------------------------------|
| Traversal | Accessing each element exactly once         |
| Searching | Finding the location of a specific element  |
| Insertion | Adding a new element at a specific position |
| Deletion  | Removing an existing element                |
| Sorting   | Arranging elements in a specific order      |
| Merging   | Combining two data structures into one      |

## 4. Algorithm Analysis

---

### What is an Algorithm?

---

An **algorithm** is a finite set of step-by-step instructions to solve a given problem.

### Properties of an Algorithm

| Property     | Description                                 |
|--------------|---------------------------------------------|
| Input        | Zero or more inputs supplied externally     |
| Output       | At least one output obtained                |
| Definiteness | Each step is clear and well-defined         |
| Finiteness   | Algorithm must terminate after finite steps |
| Correctness  | Every step must generate a correct output   |

### Time Complexity

---

Time complexity measures the amount of time an algorithm takes to run, depending on the amount of data it processes.

### Common Time Complexities (Big O Notation)

| Notation      | Name         | Example                          |
|---------------|--------------|----------------------------------|
| $O(1)$        | Constant     | Direct access to array element   |
| $O(\log n)$   | Logarithmic  | Binary Search                    |
| $O(n)$        | Linear       | Linear Search, Traversal         |
| $O(n \log n)$ | Linearithmic | Merge Sort, Quick Sort (average) |

|          |             |                             |
|----------|-------------|-----------------------------|
| $O(n^2)$ | Quadratic   | Bubble Sort, Selection Sort |
| $O(2^n)$ | Exponential | Recursive Fibonacci         |

Time Complexity Growth Rates  
(faster to slower)

$O(1)$        $<$     $O(\log n)$     $<$     $O(n)$        $<$     $O(n \log n)$     $<$     $O(n^2)$        $<$     $O(2^n)$   
 Constant    $<$    Logarithmic    $<$    Linear       $<$    Linearithmic    $<$    Quadratic    $<$    Exponential



## Space Complexity

Space complexity measures the amount of memory an algorithm uses.

### Components of Space Complexity

1. **Instruction Space:** Space to store the executable version of the program
2. **Data Space:** Space to store constants and variables
3. **Environment Space:** Space to store environment information (e.g., function call stack)

## Time-Space Trade-off

The principle that there is often a trade-off between the time an algorithm takes to execute and the memory it uses.

TIME-SPACE TRADE-OFF

-----

High Time Efficiency    $\leftrightarrow$    Low Space Efficiency  
 Low Time Efficiency     $\leftrightarrow$    High Space Efficiency

Example: Caching

- Using cache (more memory) reduces time to access data
- Accessing data from disk (less memory) takes more time

## 5. Arrays

### Definition

An **array** is a collection of similar data elements stored in contiguous memory locations and accessed using an index.

ARRAY IN MEMORY

|     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|
| 10  | 20  | 30  | 40  | 50  | 60  |
| [0] | [1] | [2] | [3] | [4] | [5] |

Base Address: 1000

### One-Dimensional Array

#### Declaration in C

```
data_type array_name[size];

int marks[5];      // Array of 5 integers
float prices[10];  // Array of 10 floats
char name[20];     // Array of 20 characters
```

#### Initialization

```
int marks[5] = {80, 85, 90, 75, 88};
int numbers[] = {1, 2, 3, 4, 5}; // Size automatically determined
```

#### Address Calculation

For an array A with base address B, element size W, and index i:

$$\text{Address}(A[i]) = B + (i * W)$$



**Example:** int arr[5] with base address 1000, size of int = 4 bytes

```
Address(arr[0]) = 1000 + (0 * 4) = 1000
Address(arr[1]) = 1000 + (1 * 4) = 1004
Address(arr[2]) = 1000 + (2 * 4) = 1008
Address(arr[3]) = 1000 + (3 * 4) = 1012
Address(arr[4]) = 1000 + (4 * 4) = 1016
```

## Multi-Dimensional Arrays

A multidimensional array is an array of arrays. The most common is the 2D array (matrix).

### Declaration of 2D Array

```
data_type array_name[row_size][column_size];

int matrix[3][4]; // 3 rows, 4 columns
```

### Address Calculation for 2D Array

#### Row-Major Order:

```
Address(A[i][j]) = B + (i * C + j) * W
Where: B = Base address, C = Number of columns, W = Element size
```

#### Column-Major Order:

```
Address(A[i][j]) = B + (j * R + i) * W
Where: B = Base address, R = Number of rows, W = Element size
```

### 2D Array Example Program

```
#include <stdio.h>

int main() {
    int matrix[3][3], i, j;

    // Input
```

```
printf("Enter elements of 3x3 matrix:\n");
for (i = 0; i < 3; i++) {
    for (j = 0; j < 3; j++) {
        scanf("%d", &matrix[i][j]);
    }
}

// Output
printf("\nMatrix:\n");
for (i = 0; i < 3; i++) {
    for (j = 0; j < 3; j++) {
        printf("%d ", matrix[i][j]);
    }
    printf("\n");
}
return 0;
}
```

## 6. Array Operations

---

### Traversal

---

Accessing each element of the array exactly once.

### Algorithm

1. Set  $i = 0$
2. Repeat steps 3-4 while  $i < n$
3. Process `arr[i]`
4.  $i = i + 1$
5. Stop

### Program

```
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int i;

    for (i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```

**Output:** 10 20 30 40 50

### Insertion

---

Inserting a new element at a specific position in the array.

### Algorithm

1. Input the element and position
2. Check if the position is valid (0 to n)
3. Shift all elements from the position to the right by one
4. Insert the new element at the position
5. Increase the size by 1

## Program

```
#include <stdio.h>

int main() {
    int arr[100] = {10, 20, 30, 40, 50};
    int n = 5, pos, element, i;

    printf("Enter element to insert: ");
    scanf("%d", &element);
    printf("Enter position (0 to %d): ", n);
    scanf("%d", &pos);

    if (pos < 0 || pos > n) {
        printf("Invalid position!\n");
        return 1;
    }

    // Shift elements right
    for (i = n; i > pos; i--) {
        arr[i] = arr[i - 1];
    }

    arr[pos] = element;
    n++;

    printf("Array after insertion: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```

## Deletion

Removing an element from a specific position in the array.

## Algorithm

1. Input the position
2. Check if the position is valid (0 to n-1)
3. Shift all elements from the position to the left by one
4. Decrease the size by 1

## Program

```
#include <stdio.h>

int main() {
    int arr[100] = {10, 20, 30, 40, 50};
    int n = 5, pos, i;

    printf("Enter position to delete (0 to %d): ", n-1);
    scanf("%d", &pos);

    if (pos < 0 || pos >= n) {
        printf("Invalid position!\n");
        return 1;
    }

    // Shift elements left
    for (i = pos; i < n - 1; i++) {
        arr[i] = arr[i + 1];
    }
    n--;

    printf("Array after deletion: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```

## 7. Strings

### Definition

A **string** is a sequence of characters terminated by a null character ('\0') in C. Strings are stored as arrays of characters.

### Declaration and Initialization

```
char str1[20] = "Hello";  
char str2[] = "World";  
char str3[10] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

### Common String Operations

| Operation   | Function           | Description                   |
|-------------|--------------------|-------------------------------|
| Length      | strlen(str)        | Returns length of string      |
| Copy        | strcpy(dest, src)  | Copies source to destination  |
| Concatenate | strcat(dest, src)  | Appends source to destination |
| Compare     | strcmp(str1, str2) | Compares two strings          |
| Reverse     | strrev(str)        | Reverses the string           |
| Uppercase   | strupr(str)        | Converts to uppercase         |
| Lowercase   | strlwr(str)        | Converts to lowercase         |

### Example Program

```
#include <stdio.h>  
#include <string.h>
```

```
int main() {  
    char str1[50] = "Hello";  
    char str2[50] = "World";  
  
    printf("Length of str1: %lu\n", strlen(str1));  
    strcat(str1, str2);  
    printf("After concatenation: %s\n", str1);  
    printf("Comparison: %d\n", strcmp(str1, "HelloWorld"));  
    strcpy(str1, "Data");  
    printf("After copy: %s\n", str1);  
    return 0;  
}
```

## Advantages of Arrays

---

- Fast random access –  $O(1)$  time complexity for accessing any element
- Cache-friendly – contiguous memory improves cache performance
- Simple to implement and use
- Efficient for storing and accessing fixed-size data

## Disadvantages of Arrays

---

- Fixed size – cannot grow or shrink dynamically
- Insertion and deletion are expensive –  $O(n)$  due to shifting
- Wasted memory if allocated size exceeds requirements
- Only homogeneous data can be stored

## 8. Quick Revision

---

### Big O Notation Summary

---

| Notation      | Name         |
|---------------|--------------|
| $O(1)$        | Constant     |
| $O(\log n)$   | Logarithmic  |
| $O(n)$        | Linear       |
| $O(n \log n)$ | Linearithmic |
| $O(n^2)$      | Quadratic    |
| $O(2^n)$      | Exponential  |

### Array Formulas

---

| Formula                                                   | Description             |
|-----------------------------------------------------------|-------------------------|
| $\text{Address}(A[i]) = B + i \times W$                   | 1D Array                |
| $\text{Address}(A[i][j]) = B + (i \times C + j) \times W$ | 2D Array (Row-Major)    |
| $\text{Address}(A[i][j]) = B + (j \times R + i) \times W$ | 2D Array (Column-Major) |



## 9. Questions

---

### Short Answer Questions (2-5 Marks)

---

1. What is a data structure?
2. What are the types of data structures?
3. What is the difference between primitive and non-primitive data structures?
4. What is an algorithm? What are its properties?
5. What is time complexity?
6. What is space complexity?
7. What is the time-space trade-off?
8. What is an array? How is it represented in memory?
9. What is sequential allocation?
10. Define the traversal operation on arrays.
11. Define the insertion operation on arrays.
12. Define the deletion operation on arrays.
13. How is a 2D array stored in memory?
14. What is the difference between row-major and column-major order?
15. What are the advantages and disadvantages of arrays?

### Long Answer Questions (10 Marks)

---

1. Explain the concept and classification of data structures with examples.
2. Explain algorithm complexity with time and space complexity examples.
3. Explain the time-space trade-off with suitable examples.
4. Explain arrays with operations: traversal, insertion, and deletion.
5. Explain multidimensional arrays with address calculation.
6. Explain row-major and column-major order for 2D arrays with examples.
7. What is an array? Write a program to insert and delete elements from an array.
8. Explain string operations with C functions.

### Objective Questions (1 Mark)

---

1. A data structure is a:

- (a) Storage of data (b) Organization of data (c) Both (d) None

**Answer: (c)**

2. Which is a primitive data structure?

- (a) Array (b) Stack (c) Integer (d) Linked List

**Answer: (c)**

3. The time complexity of linear search is:

- (a)  $O(1)$  (b)  $O(n)$  (c)  $O(n^2)$  (d)  $O(\log n)$

**Answer: (b)**

4. Arrays store data in:

- (a) Non-contiguous memory (b) Contiguous memory (c) Both (d) None

**Answer: (b)**

5. The address of  $\text{arr}[i]$  is:

- (a)  $B + i$  (b)  $B + i \times W$  (c)  $B + W$  (d)  $B \times W$

**Answer: (b)**

6. In row-major order, address of  $A[i][j]$  is:

- (a)  $B + (i \times C + j) \times W$  (b)  $B + (j \times R + i) \times W$  (c) Both (d) None

**Answer: (a)**

---

*End of Unit-1 Notes — Data Structures (HPU BCA Sem-2)*